



UFOP

Universidade Federal
de Ouro Preto

Universidade Federal de Ouro Preto – UFOP
Instituto de Ciências Exatas e Biológicas – ICEB
Departamento de Computação – DECOM
Disciplina: Engenharia de Software - BCC 322
Professor: Tiago Garcia de Senna Carneiro

Trabalho Individual - Sprint 1

Alunos: Henrique Ulhôa de Faria, Matheus Mota Gomes

Matrículas: 22.2.4088, 22.2.4101

Turma: 11

Ouro Preto, 21 de maio de 2026

Sumário

Sumário	2
1 CASOS DE USO	3
1.1 Criar um Sistema	3
1.2 Criar um Fluxo	4
1.3 Fluxo sem origem conectado a um sistema	5
1.4 Um Sistema conectado a um Fluxo	6
1.5 Sistema com um fluxo na entrada e outro na saída	7
1.6 Dois sistemas conectados por um fluxo	8
1.7 Um sistema com dois fluxos de saída	9
2 CRITÉRIOS DE ACEITAÇÃO	10
3 CENARIOS DE TESTES	11
3.1 Cenário 1 – Modelo Exponencial	11
3.1.1 Descrição do modelo (Vensim)	11
3.1.2 Pseudo-código	11
3.2 Cenário 2 – Modelo Logístico	12
3.2.1 Descrição do modelo (Vensim)	12
3.2.2 Pseudo-código	12
3.3 Cenário 3 – Modelo Complexo com 5 Sistemas e 6 Fluxos	13
3.3.1 Descrição do modelo (Vensim)	13
3.3.2 Pseudo-código	13
4 DIAGRAMA DE CLASSES	15

1 Casos de Uso

1.1 Criar um Sistema

Ator	Modelador
Descrição	O modelador cria um objeto do tipo <code>System</code> e um modelo.
Pré-condição	Nenhuma.
Fluxo principal	1. O modelador declara um objeto <code>System</code> e <code>Model</code> . 2. O modelo armazena os sistemas e fluxos adicionados. 3. O modelador roda a simulação.
Pós-condição	Um objeto <code>System</code> está disponível para ser associado a um <code>Model</code> .

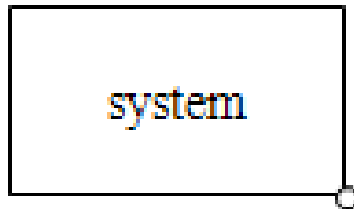


Figura 1 – Diagrama de sequência do caso de uso "Criar um Sistema".

Listing 1.1 – Class System

```

1  class System {
2  private:
3      double value;
4      string name;
5
6  public:
7      System();
8      System(const string& name, double value);
9      System(const System& other);
10     virtual ~System();
11
12     System& operator=(const System& other);
13 }

```

1.2 Criar um Fluxo

Ator	Modelador
Descrição	O modelador cria um objeto concreto derivado de Flow.
Pré-condição	Nenhuma.
Fluxo principal	1. O modelador instancia uma subclasse concreta de MyFlow. 2. Adiciona o fluxo ao modelo. 3. Roda a simulação.
Pós-condição	Um objeto Flow está pronto para ser adicionado a um Model.

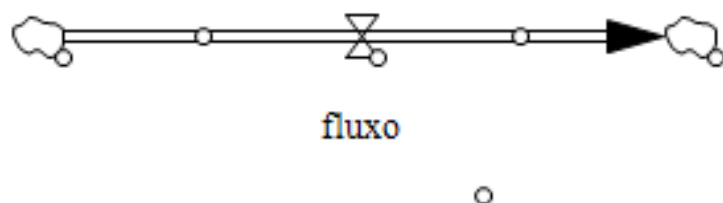


Figura 2 – Diagrama de sequência do caso de uso "Criar um Fluxo".

Listing 1.2 – Class Flow

```

1  class Flow {
2  protected:
3      string name;
4      System* source;
5      System* target;
6
7  public:
8
9      Flow();
10     Flow(const string& name,
11           System* source,
12           System* target);
13
14     Flow(const Flow& other);
15
16     virtual ~Flow();
17
18     Flow& operator=(const Flow& other);
19     // Metodo abstrato
20     virtual double execute() = 0;
21 }

```

1.3 Fluxo sem origem conectado a um sistema

Ator	Modelador
Descrição	O modelador cria um objeto Model e adiciona a ele os sistemas e fluxos que compõem o modelo de simulação.
Pré-condição	Pelo menos um System e um Flow já foram criados.
Fluxo principal	1. Chama <code>m.add(system)</code> para cada sistema. 2. Chama <code>m.add(flow)</code> para cada fluxo. 3. Chama <code>f1.setTarget(&s1)</code> para estabelecer a conexão. 4. Roda a simulação.
Pós-condição	O Model contém todos os sistemas e fluxos necessários para a simulação.

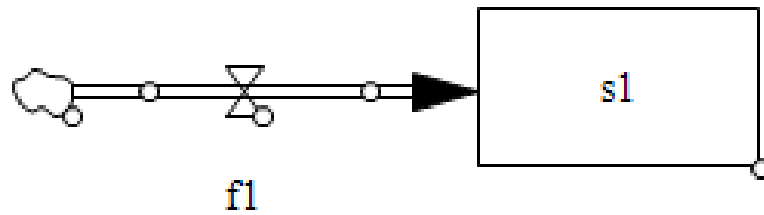


Figura 3 – Diagrama de sequência do caso de uso "Fluxo sem origem conectado a um sistema".

1.4 Um Sistema conectado a um Fluxo

Ator	Modelador
Descrição	O modelador cria um objeto <code>Model</code> e adiciona a ele os sistemas e fluxos que compõem o modelo de simulação.
Pré-condição	Pelo menos um <code>System</code> e um <code>Flow</code> já foram criados.
Fluxo principal	1. Chama <code>f1.setSource(&s1)</code> para estabelecer a conexão. 2. Chama <code>m.add(system)</code> para cada sistema. 3. Chama <code>m.add(flow)</code> para cada fluxo. 4. Roda a simulação.
Pós-condição	O <code>Model</code> contém todos os sistemas e fluxos necessários para a simulação.

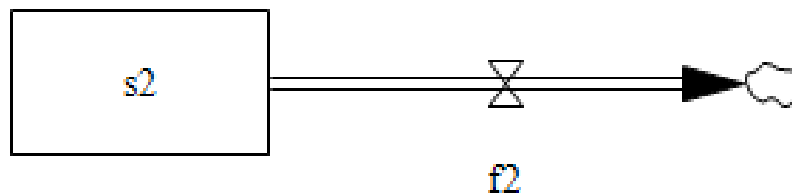


Figura 4 – Diagrama de sequência do caso de uso "Um Sistema conectado a um Fluxo".

1.5 Sistema com um fluxo na entrada e outro na saída

Ator	Modelador
Descrição	O modelador cria um objeto <code>Model</code> e adiciona a ele os sistemas e fluxos que compõem o modelo de simulação.
Pré-condição	Pelo menos um <code>System</code> e dois <code>Flow</code> já foram criados.
Fluxo principal	1. Chama <code>f1.setTarget(&s1)</code> para estabelecer a conexão. 2. Chama <code>f2.setSource(&s1)</code> para cada fluxo. 3. Chama <code>m.add(&s1)</code> para cada sistema. 4. Chama <code>m.add(&f1)</code> para cada fluxo. 5. Chama <code>m.add(&f2)</code> para cada fluxo. 6. Roda a simulação.
Pós-condição	O <code>Model</code> contém todos os sistemas e fluxos necessários para a simulação.

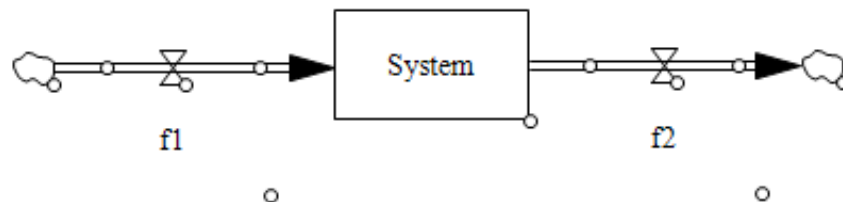


Figura 5 – Diagrama de sequência do caso de uso "Sistema com um fluxo na entrada e outro na saída".

1.6 Dois sistemas conectados por um fluxo

Ator	Modelador
Descrição	O modelador cria um objeto <code>Model</code> e adiciona a ele os sistemas e fluxos que compõem o modelo de simulação.
Pré-condição	Pelo menos dois <code>System</code> e um <code>Flow</code> já foram criados.
Fluxo principal	1. Chama <code>f1.setSource(&s1)</code> para estabelecer a conexão. 2. Chama <code>f1.setTarget(&s2)</code> para estabelecer a conexão. 3. Chama <code>m.add(&s1)</code> para cada sistema. 4. Chama <code>m.add(&s2)</code> para cada sistema. 5. Chama <code>m.add(&f1)</code> para cada fluxo. 6. Roda a simulação.
Pós-condição	O <code>Model</code> contém todos os sistemas e fluxos necessários para a simulação.

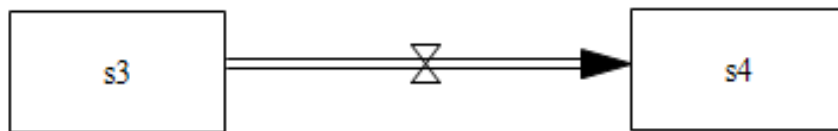


Figura 6 – Diagrama de sequência do caso de uso "Dois sistemas conectados por um fluxo".

1.7 Um sistema com dois fluxos de saída

Ator	Modelador
Descrição	O modelador cria um objeto Model e adiciona a ele os sistemas e fluxos que compõem o modelo de simulação.
Pré-condição	Pelo menos um System e dois Flow já foram criados.
Fluxo principal	1. Chama <code>f1.setSource(&s1)</code> para estabelecer a conexão. 2. Chama <code>f2.setSource(&s1)</code> para estabelecer a conexão. 3. Chama <code>m.add(&s1)</code> para cada sistema. 4. Chama <code>m.add(&f1)</code> para cada fluxo. 5. Chama <code>m.add(&f2)</code> para cada fluxo. 6. Roda a simulação.
Pós-condição	O Model contém todos os sistemas e fluxos necessários para a simulação.

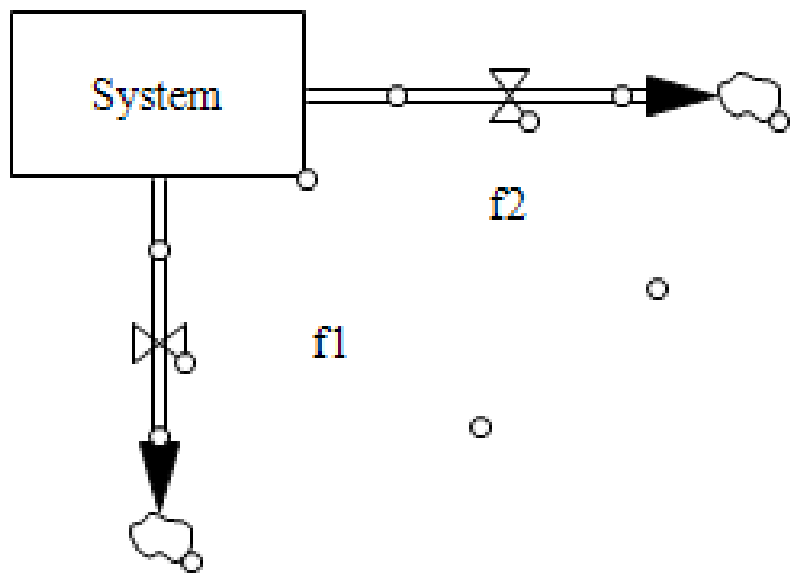


Figura 7 – Diagrama de sequência do caso de uso "Um sistema com dois fluxos de saída".

2 Critérios de Aceitação

Os critérios de aceitação foram identificados com base nos modelos de referência do enunciado e validados pelo software Vensim PLE.

CA01	Atribuição de Função ao Fluxo
Descrição	O Modelador atribui uma função anônima à equação do fluxo, através da sobrecarga de função.
Verificação	<code>executeEquation()</code> forçara o modelador a implementar a lógica específica para cada fluxo. Devido a existencia da classe abstrata <code>MyFlow</code> .

CA02	Crescimento Exponencial
Modelo	P=10, taxa de natalidade = 0,3, 100 passos de tempo.
Resultado esperado	P cresce exponencialmente sem limite superior, conforme Figura 3(a) do enunciado.
Verificação	P(100) numericamente igual ao valor do Vensim PLE.

CA03	Crescimento Logístico
Modelo	P=10, taxa = 0,3, Pmax=70, 100 passos.
Resultado esperado	P converge para ≈ 70 , conforme Figura 3(b).
Verificação	$ P(100) - 70 < 0,01$ em relação ao Vensim PLE.

3 Cenários de Testes

Os cenários de teste a seguir foram derivados diretamente dos modelos definidos no arquivo `ValidacaoMyVENSIM.mdl`, que serve como referência de validação da API. O arquivo define três grupos de modelos com `INITIAL TIME = 0`, `FINAL TIME = 100` e `TIME STEP = 1`.

Estratégia de projeto da API: cada cenário de teste foi elaborado antes da implementação, guiando o design das classes. A API deve permitir escrever os cenários abaixo de forma direta, sem configurações desnecessárias.

3.1 Cenário 1 – Modelo Exponencial

3.1.1 Descrição do modelo (Vensim)

pop1	INTEG(-exponencial, 100)
pop2	INTEG(exponencial, 200)
exponencial	$0.01 * \text{pop1}$

O fluxo `exponencial` retira de `pop1` e deposita em `pop2` a uma taxa constante de 1% de `pop1` por passo de tempo.

3.1.2 Pseudo-código

```
1 // Cenário 1: Modelo Exponencial
2 // Fonte: pop1/pop2 em ValidacaoMyVENSIM.mdl
3 // taxa = 0.01, pop1_ini = 100, pop2_ini = 200, t = [0, 100]
4
5 //main
6 System *pop1 = new System(100);
7 System *pop2 = new System(200);
8
9 Flow *exponencial = new MyFlow(); // fluxo exponencial
10 exponencial->setSource(pop1);
11 exponencial->setTarget(pop2);
12 Model m;
13 m.add(pop1);
14 m.add(pop2);
15 m.add(exponencial);
16 m.run(0, 100);
```

```

17
18 Flow *exponencial = new MyFlow(); // fluxo exponencial

```

3.2 Cenário 2 – Modelo Logístico

3.2.1 Descrição do modelo (Vensim)

p1	INTEG(-logistica, 100)
p2	INTEG(logistica, 10)
logistica	$0.01 * p2 * (1 - p2 / 70)$

O fluxo *logistica* transfere de *p1* para *p2* com crescimento limitado por $P_{max} = 70$. Conforme *p2* se aproxima de 70, o fluxo tende a zero.

3.2.2 Pseudo-código

```

1 // Cenario 2: Modelo Logistico
2 // Fonte: p1/p2 em ValidacaoMyVENSIM.mdl
3 // taxa = 0.01, Pmax = 70, p1_ini = 100, p2_ini = 200, t = [0, 100]
4
5 //main
6 System *p1 = new System(100);
7 System *p2 = new System(200);
8
9 // logistica = 0.01 * p2 * (1 - p2 / 70)
10 Flow *logistica = new MyFlow();
11 logistica->setSource(p1);
12 logistica->setTarget(p2);
13
14 Model m;
15 m.add(p1);
16 m.add(p2);
17 m.add(logistica);
18 m.run(1, 100);
19
20 // Critérios de aceitacao:
21 // p2 deve convergir para Pmax = 70
22 assert(abs(p2.getValue() - 70.0) < 1.0);
23
24 // p1 deve ter decaido (perdeu populacao para p2)
25 assert(p1.getValue() < 100.0);
26

```

```

27 // valores numericamente iguais ao Vensim PLE
28 cout << "p1(100) = " << p1.getValue() << endl;
29 cout << "p2(100) = " << p2.getValue() << endl;

```

3.3 Cenário 3 – Modelo Complexo com 5 Sistemas e 6 Fluxos

3.3.1 Descrição do modelo (Vensim)

Sistema	Equação (Vensim)	Valor inicial
Q1	INTEG(v - f - g, 100)	100
Q2	INTEG(f - r - t, 0)	0
Q3	INTEG(g + t - u, 100)	100
Q4	INTEG(u - v, 0)	0
Q5	INTEG(r, 0)	0
Fluxo	Equação	Origem → Des- tino
f	0.01 * Q1	Q1 → Q2
g	0.01 * Q1	Q1 → Q3
r	0.01 * Q2	Q2 → Q5
t	0.01 * Q2	Q2 → Q3
u	0.01 * Q3	Q3 → Q4
v	0.01 * Q4	Q4 → Q1

Este cenário valida a capacidade da API de lidar com múltiplos sistemas e fluxos simultâneos, bem como ciclos entre sistemas (Q4 → Q1 via fluxo v).

3.3.2 Pseudo-código

```

1 // Cenário 3: Modelo Complexo (Q1..Q5)
2 // Fonte: ValidacaoMyVENSIM.mdl
3 // Todos os fluxos: taxa = 0.01, t = [0, 100]
4
5 //main
6 System *Q1;
7 System *Q2;
8 System *Q3;
9 System *Q4;
10 System *Q5;
11

```

```

12 Flow *f = new MyFlow('f',Q1,Q2); // fluxo f
13 Flow *g = new MyFlow('g',Q1,Q3); // fluxo g
14 Flow *r = new MyFlow('r',Q2,Q5); // fluxo r
15 Flow *t = new MyFlow('t',Q2,Q3); // fluxo t
16 Flow *u = new MyFlow('u',Q3,Q4); // fluxo u
17 Flow *v = new MyFlow('v',Q4,Q1); // fluxo v
18
19 Model m;
20 m.add(Q1);
21 m.add(Q2);
22 m.add(Q3);
23 m.add(Q4);
24 m.add(Q5);
25 m.add(f);
26 m.add(g);
27 m.add(r);
28 m.add(t);
29 m.add(u);
30 m.add(v);
31 m.run(1, 100);
32 // Critérios de aceitacao:
33 // Q1 deve ter decaído (saem f e g, entra apenas v)
34 assert(Q1.getValue() < 100.0);
35
36 // Q5 so recebe (fluxo r), nunca perde -- deve ser maior que 0
37 assert(Q5.getValue() > 0.0);
38
39 // valores numericamente iguais ao Vensim PLE
40 cout << "Q1(100) = " << Q1.getValue() << endl;
41 cout << "Q2(100) = " << Q2.getValue() << endl;
42 cout << "Q3(100) = " << Q3.getValue() << endl;
43 cout << "Q4(100) = " << Q4.getValue() << endl;
44 cout << "Q5(100) = " << Q5.getValue() << endl;

```

Os cenários de teste a seguir foram derivados diretamente dos modelos definidos no arquivo ValidacaoMyVENSIM.mdl, que serve como referência de validação da API. O arquivo define três grupos de modelos com INITIAL TIME = 0, FINAL TIME = 100 e TIME STEP = 1.

Estratégia de projeto da API: cada cenário de teste foi elaborado antes da implementação, guiando o design das classes. A API deve permitir escrever os cenários abaixo de forma direta, sem configurações desnecessárias.

4 Diagrama de Classes

